

Assinatura: _____

Questão 1) (4,0 pontos) Considere o protocolo go-back-N (GBN). Assuma que o tamanho da janela do emissor é igual a J , e que os números de sequência variem entre 0 e S (**tamanho do espaço = $S+1$ e a aritmética é módulo $S+1$**). Suponha que no instante de tempo t , o receptor esteja esperando um pacote com número de sequência 0. Cada item deve vir acompanhado de no mínimo 1 diagrama e no mínimo 2 frases. O ACK é cumulativo (contendo número de seq. do **último pacote recebido em ordem com sucesso**). Podem ocorrer corrupções, perdas e atrasos, mas não reordenações, exceto no item b).

- a) (0,5 ponto) Seja $J=1$ e $S=2$, ou seja, os números de sequência possíveis são 0, 1 e 2. Quais os possíveis números de sequência dentro da janela do emissor, no instante de tempo t ? Justifique claramente.

O emissor pode estar com # de sequência 0 na sua janela, sem dúvidas. Ele pode também estar com # de seq. 2, porque pode não ter recebido ACK do 2. Por outro lado, 1 não é possível.

Com $J=1$, o emissor é, na prática, um stop-and-wait: ele só pode ter **um pacote pendente** de ACK por vez.

Se o receptor está esperando 0, há dois cenários possíveis:

1. **Início de ciclo:** o emissor já recebeu o ACK do pacote 2 (ACK2) e voltou a enviar o pacote 0; nesse caso, a janela contém o pacote 0.
2. **ACK atrasado:** o receptor já recebeu 0, 1 e 2 (então espera 0), mas o ACK2 ainda não chegou ao emissor; o emissor ainda está com o pacote 2 como último enviado e pendente de ACK.

Já o pacote 1 não pode estar na janela quando o receptor espera 0, pois isso exigiria que o último pacote confirmado pelo emissor fosse o 0 (logo o receptor deveria estar esperando 1, não 0).

- b) (0,5 ponto) Assuma que o canal pode reordenar pacotes. Seja $J=1$ e $S=2$. Quais os possíveis números de sequência dentro da janela do emissor, no instante de tempo t ? Justifique claramente sua resposta.

Com reordenamento de pacotes, tudo é possível.

- c) (1,0 ponto) Seja $J=2$ e $S=3$. Quais são os possíveis valores de número de sequência de ACKs dos pacotes propagando do receptor para o emissor no instante de tempo t ?

Os ACKs propagando podem ser [2,3], sem dúvidas, porque foram os dois últimos recebidos. A questão é se o 0 ou 1 também poderiam estar em trânsito. O pacote 0 certamente não pode estar em trânsito.

$0 - J - 1 = 0 - 2 - 1 = -3 = 1$ é possível

Cenário:

receptor envia ACK 1

demora para chegar

receptor reenvia ACK1

finalmente o primeiro dos dois ACKs 1 chega, o que permite que o emissor envie pacotes 2 e 3

envia ACKs 2 e 3, e nesse momento estão em trânsito ainda o reenvio do ACK1, bem como esses novos envios dos ACKs 2 e 3

- d) (1,0 ponto) Calcule a utilização do sistema quando $S=1$ e $J=1$, assumindo tempo de transmissão de pacotes de dados é igual a $1/R$, onde R é a taxa de transmissão, em pacotes por segundo, e tempo de propagação ida-e-volta igual a RTT (ou seja, o tempo de propagação de um pacote, para ir do emissor para o receptor, e voltar para o emissor, é igual a RTT). RTT aqui é **apenas propagação ida-e-volta**; tempos de processamento, enfileiramento e tempo de transmissão de ACKs são desprezíveis.

$$U = 1/R / (1/R + RTT)$$

o protocolo se comporta como **stop-and-wait**: a cada ciclo, o emissor transmite um pacote de dados

A utilização é a fração de tempo em que o link está efetivamente ocupado transmitindo dados

- e) (1,0 ponto) Explique um possível problema relacionado ao cenário $J=2$ e $S=1$ (os números de sequência possíveis são 0 e 1). Qual o menor valor de S para que o problema não ocorra? Justifique.

O problema é que pode-se confundir retransmissão com novas emissões. Precisamos que S seja maior igual a 2, ou seja, que os números de seq variem entre 0, 1 e 2. Se o tamanho da janela é 2, isso não gerará problema, porque mesmo que o emissor envie os pacotes 0 e 1, por exemplo e depois retransmita, o receptor já estará esperando o pacote 2. O receptor só irá receber o pacote 2 depois que o emissor receber o ACK do pacote 1.

O problema pode acontecer se $S = J$?

Se o receptor puder receber pacotes fora de ordem, ou seja, se ele não descartar pacotes fora de ordem, o problema pode ocorrer.

Se o receptor está esperando pacotes de 0 até $J-1$, podem estar propagando de volta ACKs dos pacotes de -1 até $-J$. Assim, a quantidade máxima de pacotes em circulação com #s de seq distintos no sistema é $2J$, e a quantidade de #s de seq precisa ser $2J$.

Note que o receptor usa janela de tamanho J . Se o receptor ignorar pacotes fora de ordem, então bastam J números de sequência para resolver o problema. Se o receptor não ignorar pacotes fora de ordem, então são necessários $2J$.

No GBN, como mostrado no livro do Kurose, como o receptor **descarta todos os pacotes fora de ordem**, ele nunca precisa distinguir pacotes “velhos” de “novos”: aceita somente o próximo número esperado. Portanto, pacotes antigos atrasados não causam confusão de entrega. A única ambiguidade possível ocorre no **emissor**,

que precisa distinguir ACKs antigos de novos. Para isso, basta garantir que a janela do emissor caiba dentro do espaço de sequência.

Questão 2) (1,0 ponto) Considere um protocolo do tipo stop-and-wait, como RDT 3.0, com números de sequência, checksums, timeouts, retransmissões e acknowledgments (ACKs). Assuma que podem ocorrer timeouts prematuros, perdas, atrasos e corrupções de pacotes, mas não reordenamentos. O receptor, ao receber qualquer pacote, sempre responde com um ACK informando o último pacote recebido com sucesso. Os ACKs podem ser duplicados. Quando o emissor recebe um ACK corrompido, ou um ACK inesperado, o que ele deve fazer? Explique os detalhes. Justifique claramente sua resposta, incluindo diagramas.

Quando emissor recebe ACK corrompido ou inesperado, ele deve esperar timeout.

Quando o emissor recebe um ACK corrompido ou com número de sequência inesperado, ele deve **ignorar esse ACK e continuar esperando** pelo ACK correto ou pelo timeout. Se o timeout expirar, ele **retransmite o último pacote enviado**.

Questão 3) (3,0 pontos) Teoria de filas. Preencha as lacunas. Justifique claramente suas respostas. *Cada lacuna vale meio ponto, com exceção da última, que vale um ponto.*

Considere um roteador, que passa por períodos ocupados e por períodos ociosos. Quando o sistema está ocupado, temos pacotes esperando na fila de espera e um pacote sendo servido. Quando o sistema está ocioso, a fila de espera e o servidor estão vazios.

Seja $I = a/R$ a intensidade de tráfego. Nesse contexto, a representa ____ (1) ____ e R representa a ____ (2) _____. Para o sistema ser estável, precisamos que ____ (3) _____. A utilização é dada por ____ (4) _____.

O número médio de pacotes no sistema, contemplando tanto períodos ocupados quanto períodos ociosos, é dado por ____ (5) _____. (você deve preencher aqui com a fórmula do número médio de itens em uma fila M/M/1, em função de I , e derivar esta fórmula passo a passo)

a representa a **taxa de chegada de pacotes** ao roteador (pacotes por segundo).

R representa a **taxa de serviço/capacidade de transmissão** do roteador (pacotes por segundo).

Muito importante notar que a e R tem a mesma unidade, porque assumimos $L = 1$. Assim, a/R precisa ser adimensional.

Para o sistema ser estável, $I < 1$.

- (1) taxa de chegada, ou vazão, porque em equilíbrio
- (2) capacidade de serviço – atenção! Não é vazão!

- (3) $I < 1$
(4) $U = I$
(5) $N = I / (1 - I)$ (demonstrar!)

Questão 4) (2,0 pontos) DNS e HTTP.

- a) Considere uma implementação padrão de um servidor DNS com cache. Essa implementação irá envolver sockets do tipo TCP ou UDP? Por que? Para popular o cache, quando o servidor for resolver requisições consultando outros servidores é melhor usar DNS iterativo ou recursivo? Por que?
- Em uma implementação padrão de servidor DNS com cache, as consultas são implementadas principalmente sobre **UDP**. O UDP é preferido porque evita o overhead de estabelecimento e encerramento de conexão, reduz a latência e é suficiente para a maioria das mensagens DNS, que são pequenas.
 - O TCP é utilizado apenas em casos especiais, como respostas muito grandes (truncadas), DNSSEC ou quando explicitamente requerido, mas o comportamento “normal” e mais frequente usa UDP.
 - Para **popular o cache**, é mais vantajoso que o servidor local faça **consultas iterativas** aos outros servidores (raiz, TLD, autoritativos).
 - No modo iterativo, o servidor vai recebendo respostas parciais (“pergunte a tal servidor”) e, a cada passo, ele pode **armazenar no cache** os registros retornados (por exemplo, servidores raiz, TLD, registros NS e A dos autoritativos), enriquecendo o seu cache com cada consulta.
 - Se o servidor simplesmente encaminhasse consultas **recursivas** para outro resolver, boa parte dessa informação intermediária ficaria armazenada apenas no cache do outro servidor, reduzindo o benefício do cache local.
- b) Considere uma implementação padrão de um servidor HTTP/1.1. Quais as vantagens e desvantagens da abordagem persistente contra não persistente? Quais das abordagens são stateful e quais stateless?

Na abordagem **persistente** (HTTP/1.1 com conexões persistentes), a mesma conexão TCP é reutilizada para **várias requisições/respostas** entre cliente e servidor.

- **Vantagens:** reduz o número de handshakes TCP, diminui a latência total na transferência de múltiplos objetos (imagens, CSS, JS, etc.), e aproveita melhor o crescimento da janela de congestionamento do TCP, o que melhora a vazão efetiva.
- **Desvantagens:** o servidor precisa manter conexões abertas por mais tempo, consumindo recursos (memória, descritores de socket); conexões ociosas podem ficar penduradas, exigindo políticas de timeout e gerenciamento mais cuidadoso.

Na abordagem **não persistente**, cada objeto (ou pequeno grupo de objetos) é transferido em uma **nova conexão TCP**, que é aberta e fechada para cada requisição principal.

- **Vantagens:** o servidor “limpa” o estado de conexão com mais frequência, simplificando o gerenciamento de recursos e evitando conexões longas e ociosas.
- **Desvantagens:** há mais overhead com múltiplos handshakes TCP e encerramentos de conexão, o que aumenta o atraso total, especialmente em páginas com muitos objetos.

Do ponto de vista da **semântica do protocolo HTTP**, tanto a forma persistente quanto a não persistente são **stateless**: cada requisição HTTP é independente, e o servidor não precisa manter contexto de aplicação entre requisições (a menos que se use cookies, sessões, etc.).

O que muda é o estado ao nível de **conexão TCP**:

- As conexões **não persistentes** mantêm um estado de transporte de curta duração (abre–responde–fecha), o que dá uma sensação de menor “estado” no servidor.
- As conexões **persistentes** mantêm o estado da conexão TCP por mais tempo, o que torna o gerenciamento de conexão mais **stateful no nível de transporte**, embora o HTTP em si continue sem estado na camada de aplicação.